



Team Debugger

Ecommerce (Demo-blaze) Automation Testing

Capstone Project



Contents

Team Members / Task Delegation	3/5	Project Folder Structure	10-11
Project Overview	4	BDD Feature Example	12
Problem Statement	6	Page Object Model (POM)	13
Objectives	7	Execution and Reports	14-15
Tech Stack	8	Challenges and Learning	16
Framework Architecture	9	Conclusion	17

Team Members

Akash Kumar

Punjab Technical University, Jalandhar(Punjab)
kumarakash1060@gmail.com

Praneesh Sharma

Mumbai
Kalinga Institute of Industrial Technology, Bhubaneswar
praneeshsharma30@gmail.com

Alok Ranjan

Maulana Abul Kalam Azad University of Technology, Kolkata
alok08717@gmail.com

Ayush Aryan

Ranchi
Kalinga Institute of Industrial Technology, Bhubaneswar
ayusharyan90319@gmail.com

Pratyush Rath

Bhubaneshwar,
Kalinga Institute of Industrial Technology,
pravasrath56@gmail.com

Akshat Nogaraiya

GLA University, Mathura
a.nogaraiya003@gmail.com



Project Overview

Demo-blaze is a web-based e-commerce application used for end-to-end automation testing.

The project automates critical shopping workflows using Selenium WebDriver.

Python is the primary language used for scripting the automation suite.

Pytest-BDD framework is utilized to write human-readable Gherkin test scenarios.

Page Object Model (POM) is implemented for reusable, modular, and maintainable code.

The framework optimizes test execution speed, coverage, and overall reporting accuracy.



Demo-blaze

E-Commerce Test Application

Selenium

Python

Pytest-BDD

POM

Task Delegation

Team Member	Module / Role Assigned
Akash Kumar	User Navigation ,Product Functionality And Reporting
Praneesh Sharma	Add to Cart and Remove from Cart Validation
Pratyush Rath	Place Order and Order Confirmation
Ayush Aryan	Homepage and Navigation Management
Alok Ranjan	Authentication and User Management
Akshat Nogaraiya	Negative Authorization and Error Handling

Problem Statement

The Core Challenge

E-commerce platforms require **accurate and robust validation** of user navigation and top header functions.

Manual Testing Limitations

- Time-consuming for continuous regression
- Prone to human errors in critical flows
- Difficult to scale across categories

Impact of Errors

- User frustration & site abandonment
- Broken product discovery
- Cart access & modal interaction issues

Strategic Solution

Need for a robust **BDD automation framework** to:

- Validate category grid & availability
- Ensure correct homepage structure
- Verify header links & search/cart

Objectives



Automated Validation

Replace manual checks with high-speed scripts for critical e-commerce journeys.



BDD Implementation

Use Gherkin syntax to ensure test logic is readable for non-technical stakeholders.



POM Architecture

Design a clean Page Object Model to isolate UI elements and minimize maintenance.



Parallel Execution

Leverage parallel processing to execute browser instances and cut cycle time.



Visual Reporting

Generate HTML dashboards for instant quality tracking and project lead insights.

Tech Stack



Core Language
Python



Automation Engine
Selenium WebDriver



Testing Framework
Pytest



Version Control
Git and GitHub

BDD

Pytest-BDD (Gherkin)

Parallel Engine

Pytest-Xdist

Reporting Engine

Pytest-HTML

Design Pattern

POM Architecture

Framework Architecture





Project Folder Structure

Folder/FilePurpose

features/ Contains Gherkin feature files written using pytest-bdd

pages/ Contains Page Object Model (POM) classes with Selenium locators and reusable methods

tests/ Contains pytest-bdd step definitions and test execution files

reports/ Stores generated HTML test execution reports

Folder/FilePurpose

utils/ Contains reusable utility/helper files such as driver setup

conftest.py/ Contains shared pytest fixtures and Selenium WebDriver setup

requirements.txt/ Contains all Python project dependencies

.gitignore/ Specifies files and folders ignored by Git

Project Folder Structure

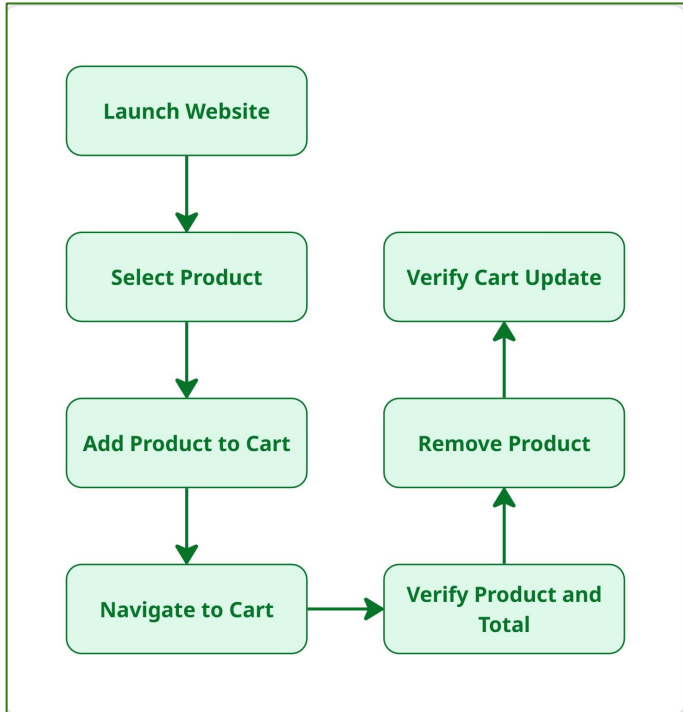
```
FEATURES
├── cart_add.feature
├── cart_remove.feature
├── categories.feature
├── invalid_login.feature
├── login.feature
├── navigation_home.feature
├── order_confirmation.feature
├── product_details.feature
├── signup_duplicate.feature
├── signup.feature
└── top_navigation_bar.feature
```

```
PAGES
├── base_page.py
├── cart_page.py
├── duplicate_signup.py
├── home_page.py
├── invalid_login.py
├── login_page.py
├── navigation_home.py
├── order_confirmation.py
├── product_page.py
└── signup_page.py
```

```
TESTS
├── test_cart.py
├── test_categories.py
├── test_login.py
├── test_navigation.py
├── test_negative_auth.py
├── test_order.py
├── test_product_detail.py
├── test_signup.py
└── test_top_navigation.py
```

```
REPORTS / CONFIG
├── reports/
│   ├── report.html
│   ├── .gitignore
│   └── assets/
│       └── style.css
├── utils/
│   └── drivers.py
├── conftest.py
├── requirements.txt
├── README.md
└── .gitignore
```

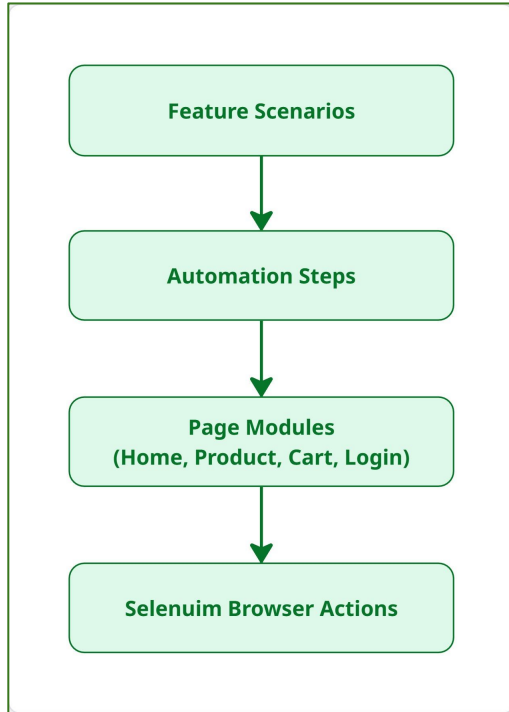
BDD Feature Example – Cart Management



The Cart Management module validates whether users can successfully add and remove products from the shopping cart.

- Simulates real user shopping behavior
- Validates end-to-end cart functionality
- Covers product addition and removal workflows
- Ensures cart total updates dynamically
- Verifies cart state after each user action
- Improves reliability of core e-commerce operations

Page Object Model (POM)



The framework was designed using the Page Object Model (POM) architecture to improve modularity and maintainability.

- Organizes automation into reusable page modules
- Separates application flow from browser interactions
- Improves framework readability and scalability
- Simplifies debugging and future enhancements
- Reduces duplication across test scenarios
- Enables smooth collaboration between team members

Execution and Reports

report.html

Report generated on 22-May-2026 at 23:26:41 by [pytest-html](#) v4.2.0

Environment

Python	3.14.4
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 9.0.3 • pluggy: 1.6.0
Plugins	• allure-pytest: 2.16.0 • bdd: 8.1.0 • html: 4.2.0 • metadata: 3.1.1 • xdist: 3.8.0

28

Total Tests

27

Passed

1

Failed

0

Errors

Pass Rate: 96.42%



Execution and Reports

Summary

28 tests took 00:08:33.

(Un)check the boxes to filter the results.

☒ 1 Failed, ☒ 27 Passed, ☒ 0 Skipped, ☒ 0 Expected failures, ☒ 0 Unexpected passes, ☒ 0 Errors, ☒ 0 Reruns ☒ 0 Retried,

[Show all details](#) / [Hide all details](#)

Result	Test	Duration	Links
Failed	tests/test_categories.py::test_verify_monitors_category	00:00:12	
Passed	tests/test_cart.py::test_add_a_single_product_to_the_cart	00:00:25	
Passed	tests/test_cart.py::test_add_multiple_products_to_the_cart	00:00:26	
Passed	tests/test_cart.py::test_verify_cart_total_after_adding_a_product	00:00:21	
Passed	tests/test_cart.py::test_remove_a_product_from_the_cart	00:00:33	
Passed	tests/test_cart.py::test_verify_cart_total_after_removing_a_product	00:00:34	
Passed	tests/test_cart.py::test_remove_all_products_from_the_cart	00:00:33	
Passed	tests/test_categories.py::test_verify_phones_category	00:00:11	
Passed	tests/test_categories.py::test_verify_laptops_category	00:00:12	
Passed	tests/test_login.py::test_successful_login	00:00:15	
Passed	tests/test_logout.py::test_verify_user_logout	00:00:17	
Passed	tests/test_navigation.py::test_verify_the_homepage_structure	00:00:10	
Passed	tests/test_navigation.py::test_click_on_categories_menu[Phones-Samsung galaxy s8-Nokia lumia 1520]	00:00:14	
Passed	tests/test_navigation.py::test_click_on_categories_menu[Laptops-Sony vaio i5-MacBook air]	00:00:12	
Passed	tests/test_navigation.py::test_click_on_categories_menu[Monitors-Apple monitor 24-ASUS Full HD]	00:00:12	
Passed	tests/test_negative_auth.py::test_verify_error_messages_for_invalid_login_attempts[non_existent_user-wrong_pass123-User does not exist.]	00:00:13	
Passed	tests/test_negative_auth.py::test_verify_error_messages_for_invalid_login_attempts[admin-incorret_pass-Wrong password.]	00:00:12	
Passed	tests/test_negative_auth.py::test_verify_error_message_when_registering_an_existing_user	00:00:13	
Passed	tests/test_order.py::test_user_places_order_successfully	00:00:22	
Passed	tests/test_order.py::test_verify_order_confirmation_popup	00:00:21	
Passed	tests/test_product_detail.py::test_view_product_information	00:00:16	
Passed	tests/test_signup.py::test_successful_signup[-]	00:00:21	
Passed	tests/test_top_navigation.py::test_verify_navigation_links_that_open_homepage_modals[Contact-New message]	00:00:42	
Passed	tests/test_top_navigation.py::test_verify_navigation_links_that_open_homepage_modals[About us-About us]	00:00:11	

Challenges & Learning

Technical Challenges

Dynamic Elements & Modals

Managing asynchronous UI updates and alert pop-ups without fragile hardcoded sleeps.

BDD Step Complexity

Ensuring strict regex matching between Gherkin .feature syntax and Python step-definition glue code.

Parallel Test Isolation

Preventing session collisions and state pollution during concurrent pytest-xdist execution.

Key Learnings & Solutions

Explicit Wait Mastery

Implemented robust Selenium WebDriverWait to smoothly handle dynamic AJAX elements.

Advanced POM Structure

Built a scalable hierarchy with a BasePage to cleanly separate locators from test logic.

Parallel Best Practices

Designed stateless test cases with dynamic browser fixture allocation for reliable testing.



Conclusion

- | | |
|---|--|
| ❖ | Successfully implemented Selenium automation testing using Python and Pytest-BDD |
| ❖ | Applied Page Object Model (POM) design pattern for scalable, reusable, and maintainable automation code. |
| ❖ | Automated major user workflows including Login, Product Selection, Add to Cart, and Order Placement. |
| ❖ | Developed BDD feature files and step definitions for better readability and business-driven testing. |
| ❖ | Generated HTML test reports and improved understanding of test execution and result analysis. |
| ❖ | Implemented positive and negative test scenarios to validate application behavior. |
| ❖ | Enhanced practical knowledge of Selenium WebDriver, Pytest framework . |



Thank You!

For your time. We are ready to answer any questions regarding the automation framework.